

```
</> Bash
```

```
sudo apt install python3.12-venv
```

Enter your Ubuntu password when prompted.

After that, recreate the virtual environment because the failed `.venv` may be incomplete:

```
</> Bash
```

```
cd ~/csce465-agentsec  
rm -rf .venv  
python3 -m venv .venv  
source .venv/bin/activate
```

Then continue with the homework setup:

```
</> Bash
```

```
python -m pip install --upgrade pip  
python -m pip install cryptography==49.0.0 pytest==9.1.1
```

That matches the environment setup required by the assignment.  HW2-2

If `sudo apt install python3.12-venv` says the package cannot be found, run:

```
</> Bash
```

```
python3 --version
```

and send me the output, because then we should install the `venv` package matching your actual Python version.

Okay im ready for task 1 walk me through

Absolutely. Task 1 is meant to show **why AES-CTR encryption by itself is not enough**. You'll intentionally build an insecure version, then demonstrate two problems:

- an attacker can modify the encrypted command without knowing the key
- the same ciphertext can be replayed and accepted twice

That is exactly what the assignment asks you to demonstrate in `baseline_ctr.py`.  HW2-2

Start in your homework folder:

</> Bash

```
cd ~/csce465-agentsec/hw2
```

Then create the file:

</> Bash

```
nano baseline_ctr.py
```

Paste this starter version:

</> Python

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
import os
```

```
PLAINTEXT = b'{"action":"READ","path":"notes.txt"}'
```

```
def encrypt_ctr(key, iv, plaintext):
```

```
    cipher = Cipher(
        algorithms.AES(key),
        modes.CTR(iv)
    )
```

```
    encryptor = cipher.encryptor()
```

```
    ciphertext = encryptor.update(plaintext) + encryptor.finalize()
```

```
    return ciphertext
```

```
def decrypt_ctr(key, iv, ciphertext):
```

Save with:

</> Plain text

```
Ctrl+O
Enter
```

Ctrl+X

Then run:

</> Bash

```
python baseline_ctr.py
```

You should see something like:

</> Plain text

Original plaintext:

```
{"action": "READ", "path": "notes.txt"}
```

Ciphertext:

```
a90f3c...
```

Receiver gets original message:

Receiver processed:

```
{"action": "READ", "path": "notes.txt"}
```

At this point, all you have done is AES-CTR encryption/decryption.

The important thing to understand is that CTR behaves conceptually like this:

</> Plain text

```
ciphertext = plaintext XOR keystream
```

or:

</> Plain text

$$C = P \text{ XOR } S$$

where:

- C = ciphertext
- P = plaintext
- S = AES-generated keystream

If an attacker changes C , that causes a predictable change in P .

For example, your plaintext contains:

</> Plain text

READ

The assignment wants you to change it to another **equal-length value** without knowing the key.

 HW2-2

Let's change:

</> Plain text

READ

into:

</> Plain text

SEND

Both are four bytes.

Add this function above `main()` :

</> Python

```
def relay_modify(ciphertext):  
    original = b"READ"  
    modified = b"SEND"  
  
    offset = PLAINTEXT.index(original)  
  
    tampered = bytearray(ciphertext)  
  
    for i in range(len(original)):  
        tampered[offset + i] ^= original[i] ^ modified[i]  
  
    return bytes(tampered)
```

This line is the key:

</> Python

```
tampered[offset + i] ^= original[i] ^ modified[i]
```

Why?

CTR originally gives us:

</> Plain text

$$C = P \text{ XOR } S$$

We want the decrypted result to become:

</> Plain text

$$P'$$

So the attacker computes:

</> Plain text

$$C' = C \text{ XOR } P \text{ XOR } P'$$

Then when the receiver decrypts:

</> Plain text

$$C' \text{ XOR } S$$

you get:

</> Plain text

$$(C \text{ XOR } P \text{ XOR } P') \text{ XOR } S$$

Since:

</> Plain text

```
C = P XOR S
```

that becomes:

```
</> Plain text
```

```
(P XOR S) XOR P XOR P' XOR S
```

The duplicate P s and S s cancel:

```
</> Plain text
```

```
P XOR P = 0
```

```
S XOR S = 0
```

leaving:

```
</> Plain text
```

```
P'
```

So the attacker can force:

```
</> Plain text
```

```
READ
```

to become:

```
</> Plain text
```

```
SEND
```

without ever learning the AES key.

Now update `main()` to demonstrate this:

```
</> Python
```

```
def main():
    key = os.urandom(32)
```

```
iv = os.urandom(16)

ciphertext = encrypt_ctr(key, iv, PLAINTEXT)

print("Original plaintext:")
print(PLAINTEXT.decode())

print("\nCiphertext:")
print(ciphertext.hex())

print("\nReceiver gets original message:")
receiver(key, iv, ciphertext)

tampered_ciphertext = relay_modify(ciphertext)

print("\nReceiver gets modified ciphertext:")
receiver(key, iv, tampered_ciphertext)
```

Run:

</> Bash

```
python baseline_ctr.py
```

You should now see:

</> Plain text

```
Original plaintext:
{"action": "READ", "path": "notes.txt"}

Receiver gets original message:
Receiver processed:
{"action": "READ", "path": "notes.txt"}

Receiver gets modified ciphertext:
Receiver processed:
{"action": "SEND", "path": "notes.txt"}
```

That is the first attack.

The assignment also specifically says to **show the XOR relationship between the original and modified bytes**.

 HW2-2

We can make that explicit.

Modify `relay_modify()` to this:

</> Python

```
def relay_modify(ciphertext):
    original = b"READ"
    modified = b"SEND"

    offset = PLAINTEXT.index(original)

    tampered = bytearray(ciphertext)

    print("\nXOR changes:")

    for i in range(len(original)):
        delta = original[i] ^ modified[i]

        print(
            chr(original[i]),
            "->",
            chr(modified[i]),
            "XOR delta:",
            hex(delta)
        )
```

Now it'll print something like:

</> Plain text

```
XOR changes:
R -> S XOR delta: 0x1
E -> E XOR delta: 0x0
A -> N XOR delta: 0xf
D -> D XOR delta: 0x0
```

Notice that some characters don't change.

For example:

</> Plain text

```
E -> E
```

SO:

</> Plain text

```
E XOR E = 0
```

and no ciphertext modification is required at that position.

Now you need the second demonstration: **replay**.

The receiver currently has no memory of which messages it already processed. Therefore an attacker can capture:

</> Plain text

```
ciphertext
```

and send it multiple times.

Add this to the bottom of `main()` :

</> Python

```
print("\nReplay attack:")

receiver(key, iv, ciphertext)
receiver(key, iv, ciphertext)
```

Now the entire `main()` should look roughly like:

</> Python

```
def main():
    key = os.urandom(32)
    iv = os.urandom(16)

    ciphertext = encrypt_ctr(key, iv, PLAINTEXT)

    print("Original plaintext:")
    print(PLAINTEXT.decode())

    print("\nCiphertext:")
    print(ciphertext.hex())

    print("\nReceiver gets original message:")
```

```
receiver(key, iv, ciphertext)

tampered_ciphertext = relay_modify(ciphertext)

print("\nReceiver gets modified ciphertext:")
receiver(key, iv, tampered_ciphertext)
```

Run it again:

</> Bash

```
python baseline_ctr.py
```

Your output should demonstrate three things:

</> Plain text

Original:

```
{"action": "READ", "path": "notes.txt"}
```

Tampered:

```
{"action": "SEND", "path": "notes.txt"}
```

Replay:

```
{"action": "READ", "path": "notes.txt"}
```

```
{"action": "READ", "path": "notes.txt"}
```

The replay works because there's currently nothing like:

</> Plain text

sequence number

message ID

nonce tracking

that tells the receiver:

I've already processed this message.

And there's no MAC, so there's nothing that tells the receiver:

Someone modified this ciphertext.

That is the whole lesson of Task 1: **confidentiality does not automatically provide integrity or replay protection.**  HW2-2

For your report, you'll need to explain three things. A good outline, which you should rewrite in your own words, is:

I implemented AES-256 in CTR mode to encrypt the fixed command `{"action": "READ", "path": "notes.txt"}`. The implementation intentionally did not use a MAC or sequence number.

Then for what happened:

I modified the ciphertext corresponding to `READ` so that it decrypted to `SEND`. I calculated the required modification using `original_byte XOR desired_byte` and XORed that value with the ciphertext. The receiver accepted the modified command even though the attacker did not know the AES key. I also replayed the original ciphertext and the receiver processed the command both times.

Then root cause:

AES-CTR provides confidentiality but does not provide integrity. Because CTR effectively XORs plaintext with a keystream, predictable changes to ciphertext produce predictable changes to plaintext. The protocol also had no sequence numbers or replay tracking, so previously valid ciphertexts could be accepted multiple times.

The assignment specifically asks for screenshots or pasted output demonstrating the experiment, so once your script works, **take a screenshot of the terminal showing the original command, modified command, XOR output, and replay.**  HW2-2

Your completed Task 1 structure should therefore be:

 Plain text

```
baseline_ctr.py
    encrypt_ctr()
    decrypt_ctr()
    receiver()
    relay_modify()
    main()
```

If you paste your `baseline_ctr.py` here after you write it, I can check it against the assignment before you move on to Task 2.

Im allowed to use AI in my work but I need to write everything on my own how do i learn how to make this python file

The best way to learn this without having AI write the assignment for you is to build `baseline_ctr.py` in small pieces and make sure you understand each piece before moving on. For Task 1, the assignment only requires you to encrypt the fixed command with AES-CTR, modify the ciphertext so `READ` becomes another equal-length value, show the XOR relationship, and replay the ciphertext.  HW2-2

I would learn it in this order.

1. First understand what you are building

Your program has four conceptual jobs:

Plain text

```
plaintext
  ↓
encrypt
  ↓
ciphertext
  ↓
attacker modifies ciphertext
  ↓
decrypt
  ↓
modified plaintext
```

Then separately:

Plain text

```
ciphertext
  ↓
receiver
  ↓
receiver again
```

That second part demonstrates replay.

You do **not** need to understand all of cryptography before coding this. For Task 1, you mainly need to understand Python functions, bytes, XOR, and how the `cryptography` library is called.

2. Learn Python bytes

Your command is:

</> Python

```
b'{"action": "READ", "path": "notes.txt"}'
```

The `b` means this is a **bytes object**, not a normal Python string.

Try this yourself in Python:

</> Bash

```
python
```

Then:

</> Python

```
message = b"READ"

print(message)
print(message[0])
print(chr(message[0]))
```

You should notice:

</> Plain text

```
message[0]
```

doesn't give `"R"`. It gives the integer value representing `R`.

Try:

</> Python

```
print(ord("R"))
print(ord("E"))
print(ord("A"))
print(ord("D"))
```

That is useful because XOR works on integers.

3. Learn XOR before touching encryption

Python's XOR operator is:

</> Python

^

Try:

</> Python

```
a = ord("R")
b = ord("S")

difference = a ^ b

print(difference)
print(hex(difference))
```

Then verify that XORing that difference with R produces S :

</> Python

```
result = a ^ difference

print(result)
print(chr(result))
```

You should get:

</> Plain text

S

This is basically the central idea behind your Task 1 modification.

A very useful property is:

</> Plain text

$$A \oplus B \oplus B = A$$

because XORing something twice cancels it out.

4. Learn what AES-CTR is doing conceptually

You do **not** implement AES yourself. The assignment explicitly tells you to use maintained library implementations.  HW2-2

You can think of CTR mode as producing a secret stream of bytes:

</> Plain text

```
plaintext:  P
keystream:  K
           ↓ XOR
ciphertext:  C
```

Mathematically:

</> Plain text

$$C = P \oplus K$$

And decrypting is:

</> Plain text

$$P = C \oplus K$$

Because:

</> Plain text

$$P \oplus K \oplus K = P$$

That explains why encryption and decryption look nearly identical internally.

The vulnerability you're demonstrating happens because if an attacker changes the ciphertext by some difference D :

```
</> Plain text
```

```
C' = C XOR D
```

then the decrypted plaintext becomes:

```
</> Plain text
```

```
P' = P XOR D
```

So if you know the original character and the character you want:

```
</> Plain text
```

```
D = original XOR desired
```

you can cause a predictable change.

5. Learn the crypto library separately

Before building the whole assignment, make a tiny throwaway file like:

```
</> Bash
```

```
nano test_aes.py
```

Start only with the imports:

```
</> Python
```

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
import os
```

Then generate a key:

```
</> Python
```

```
key = os.urandom(32)
```

Ask yourself:

Why 32 bytes?

Because:

</> Plain text

```
32 bytes × 8 bits = 256 bits
```

so this gives you an AES-256 key.

Then generate an IV:

</> Python

```
iv = os.urandom(16)
```

AES works with 16-byte blocks, so CTR uses a 16-byte initial counter value here.

Print them:

</> Python

```
print(key)
print(iv)
```

Then try:

</> Python

```
print(key.hex())
print(iv.hex())
```

Hex is just easier for humans to read.

6. Build encryption by following the library API

The library pattern is:

</> Python

```
cipher = Cipher(
    algorithms.AES(key),
    modes.CTR(iv)
)
```

You're telling Python:

Create an AES cipher using this key, operating in CTR mode with this IV.

Then:

</> Python

```
encryptor = cipher.encryptor()
```

This creates something capable of encrypting.

Then:

</> Python

```
ciphertext = encryptor.update(message) + encryptor.finalize()
```

Think of:

</> Python

```
update(...)
```

as:

process these bytes

and:

</> Python

```
finalize()
```

as:

finish the encryption operation.

So instead of memorizing this blindly, try writing this small program yourself from scratch:

</> Python

```
message = b"hello"  
key = ...  
iv = ...  
  
cipher = ...  
encryptor = ...  
  
ciphertext = ...  
  
print(ciphertext)
```

If that works, you've learned the core of encryption.

7. Then learn decryption

CTR uses the same key and IV to recover the message.

The structure is nearly identical:

</> Python

```
cipher = Cipher(  
    algorithms.AES(key),  
    modes.CTR(iv)  
)  
  
decryptor = cipher.decryptor()  
  
plaintext = decryptor.update(ciphertext) + decryptor.finalize()
```

Try:

</> Python

```
print(plaintext)
```

and make sure you recover:

</> Plain text

```
b'hello'
```

At that point, don't continue until you understand:

</> Plain text

```
message
→ encryption
→ ciphertext
→ decryption
→ original message
```

8. Turn encryption into a function yourself

Once the experiment works, ask yourself:

What values does encryption need?

Answer:

</> Plain text

```
key
iv
plaintext
```

So your function should conceptually look like:

</> Python

```
def encrypt_ctr(...):
    ...
    return ...
```

Don't copy my implementation from earlier. Try writing the inside yourself using the steps you just practiced.

Then test it:

</> Python

```
ciphertext = encrypt_ctr(key, iv, message)
print(ciphertext)
```

Do the same for:

</> Python

```
def decrypt_ctr(...):
```

This is much better for learning because you're converting something you understand into a reusable function.

9. Learn how to find READ

Now use the actual assignment command:

```
</> Python
```

```
message = b'{"action":"READ","path":"notes.txt"}'
```

Try:

```
</> Python
```

```
print(message.index(b"READ"))
```

You'll get the starting position of READ .

You can test:

```
</> Python
```

```
offset = message.index(b"READ")
```

```
print(message[offset])
```

```
print(chr(message[offset]))
```

You should get:

```
</> Plain text
```

```
R
```

That tells you exactly which ciphertext byte corresponds to the R .

CTR is useful for this demonstration because ciphertext byte positions line up with plaintext byte positions.

10. Practice changing one byte first

Don't immediately try changing the whole word.

Start with:

</> Plain text

R → S

Calculate:

</> Python

```
difference = ord("R") ^ ord("S")
```

Then ciphertext is initially immutable Python bytes , so make a modifiable copy:

</> Python

```
tampered = bytearray(ciphertext)
```

Now change one ciphertext byte at the location corresponding to R :

</> Python

```
tampered[offset] ^= difference
```

Turn it back into bytes if necessary:

</> Python

```
tampered = bytes(tampered)
```

Then decrypt it.

Your goal is to discover:

</> Plain text

```
READ
```

has become:

```
</> Plain text
```

```
SEAD
```

If you can make that happen and explain why, you genuinely understand the attack.

11. Then generalize from one character to a word

Once one character works, compare:

```
</> Python
```

```
original = b"READ"  
desired = b"SEND"
```

Try inspecting:

```
</> Python
```

```
for i in range(len(original)):  
    print(original[i], desired[i])
```

Then:

```
</> Python
```

```
for i in range(len(original)):  
    difference = original[i] ^ desired[i]  
    print(hex(difference))
```

Now you can probably figure out yourself how to modify:

```
</> Python
```

```
tampered[offset + i]
```

for each byte.

That's the part I'd strongly recommend writing yourself rather than copying.

12. Finally understand replay

Replay is actually much easier.

Suppose you have:

</> Python

```
receiver(ciphertext)
```

What happens if you simply do:

</> Python

```
receiver(ciphertext)
receiver(ciphertext)
```

?

If your receiver has no record of previously processed messages, it can't distinguish:

</> Plain text

```
legitimate first transmission
```

from:

</> Plain text

```
attacker sending the exact same ciphertext again
```

That's the second weakness the assignment wants you to show.  HW2-2

A good learning workflow

I recommend you build Task 1 in these checkpoints:

</> Plain text

```
Checkpoint 1
Encrypt "hello"
```

Checkpoint 2

Decrypt "hello"

Checkpoint 3

Turn encryption/decryption into functions

Checkpoint 4

Use the homework JSON message

Checkpoint 5

Locate READ

Checkpoint 6

Modify only R → S

Checkpoint 7

Modify READ → SEND

If you do it this way, by the end you won't just have working code—you'll be able to explain every part of it, which is especially important because your assignment says you remain responsible for understanding and verifying AI-assisted work.  HW2-2

Start with **Checkpoint 1 only**. Write a tiny Python program yourself that generates a 32-byte key, a 16-byte IV, and encrypts `b"hello"` using AES-CTR. Paste what you write here, even if it doesn't work, and I'll teach you from your code rather than giving you the finished homework.

[show me the final product](#)

Here's a complete `baseline_ctr.py` for Task 1 that satisfies the assignment's requirements: encrypt the fixed command with AES-CTR, modify `READ` to another equal-length value without the key, show the XOR relation, and replay the ciphertext.  HW2-2

`</>` Python

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
import os
```

```
PLAINTEXT = b'{"action":"READ","path":"notes.txt"}'
```

```
def encrypt_ctr(key, iv, plaintext):
    cipher = Cipher(
        algorithms.AES(key),
```

```

        modes.CTR(iv)
    )

    encryptor = cipher.encryptor()
    ciphertext = encryptor.update(plaintext) + encryptor.finalize()

    return ciphertext

def decrypt_ctr(key, iv, ciphertext):
    cipher = Cipher(

```

Run it with:

</> Bash

```
python baseline_ctr.py
```

You should get output conceptually like:

</> Plain text

```

Original plaintext:
{"action":"READ","path":"notes.txt"}

--- Normal message ---
Receiver processed:
{"action":"READ","path":"notes.txt"}

XOR relationship:
R -> S | XOR delta = 0x1
E -> E | XOR delta = 0x0
A -> N | XOR delta = 0xf
D -> D | XOR delta = 0x0

--- Modified message ---
Receiver processed:
{"action":"SEND","path":"notes.txt"}

--- Replay attack ---
First delivery:
Receiver processed:
{"action":"READ","path":"notes.txt"}

```

The three parts you should be able to explain yourself are:

- `encrypt_ctr()` and `decrypt_ctr()` use AES in CTR mode.